DEVELOPING MACHINE LEARNING SOLUTIONS

INTRODUCCIÓN.

En este curso de machine learning, aprenderá sobre el ciclo de vida de machine learning y sobre cómo utilizar los servicios de AWS en cada etapa. Además, descubrirá las diversas fuentes de modelos de machine learning y aprenderá técnicas para evaluar su rendimiento. También comprenderá la importancia de las operaciones de machine learning (machine learning operations, MLOps) para optimizar el desarrollo y la implementación de los proyectos de machine learning.



DESARROLLO DE SOLUCIONES DE ML

CICLO DE VIDA DEL DESARROLLO DE MACHINE LEARNING

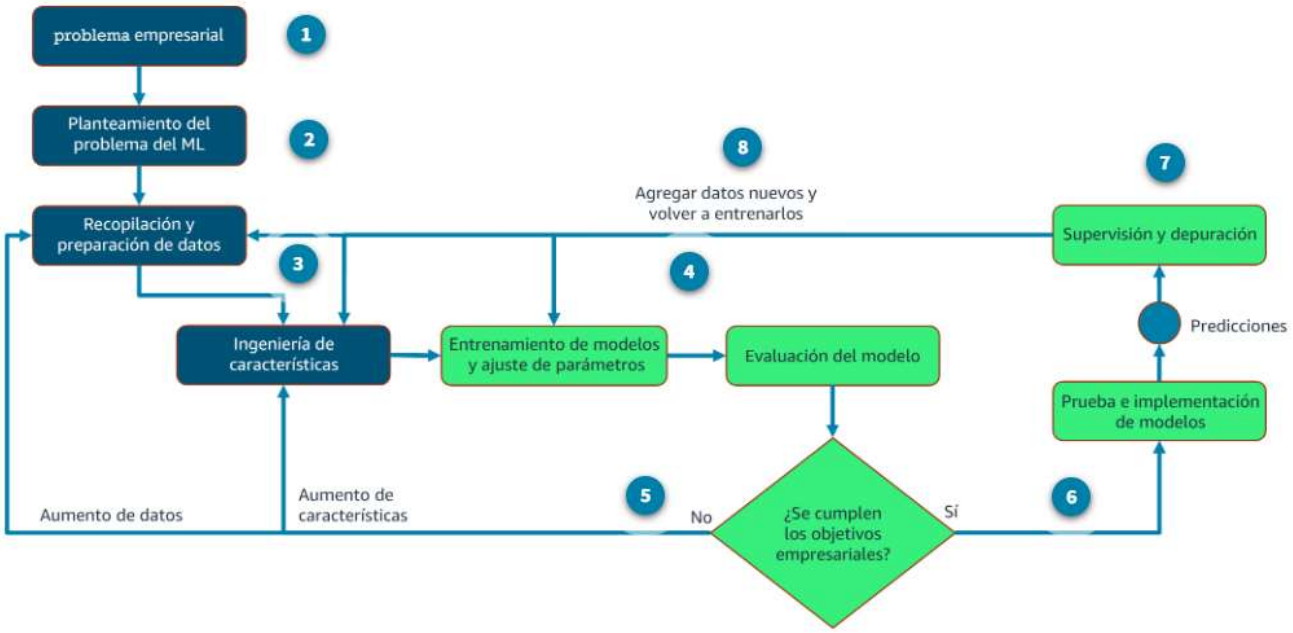
Ciclo de vida del desarrollo de ML:

Con "ciclo de vida del machine learning (ML)", se hace referencia al proceso integral de desarrollo, implementación y mantenimiento de modelos de machine learning.

El proceso integral del ciclo de vida del machine learning incluye las siguientes fases:

- identificación de objetivos empresariales,
- planteamiento del problema del ML,
- procesamiento de datos (recopilación y procesamiento previo de datos, e ingeniería de características),
- desarrollo de modelos (entrenamiento, ajuste y evaluación),
- implementación de modelos (inferencia y predicción),
- supervisión de modelos, y
- reentrenamiento de modelos.

En la siguiente ilustración, se muestra cómo funcionan las fases en conjunto. Para obtener más información al respecto, haga clic en cada uno de los marcadores numerados.



1. **Definición de objetivos empresariales:**

   El ML comienza con un objetivo empresarial. Las partes interesadas de la empresa definen el valor, el presupuesto y los criterios de éxito. Definir los criterios de éxito o los indicadores de rendimiento clave (KPI) es fundamental para la carga de trabajo de ML.

2. **Planteamiento del problema del ML:**

   La formulación del problema implica articular el problema empresarial y convertirlo en uno de machine learning.

   Los científicos de datos, los ingenieros de datos y los arquitectos de ML trabajan con los expertos en la materia (SME) de la línea de negocio a fin de determinar si es apropiado usar ML para resolver el problema empresarial. En esta fase, los equipos pueden trabajar en el descubrimiento. Se encargarán de determinar si cuentan con los datos y las habilidades adecuadas, entre otros recursos, para brindar con éxito la solución empresarial.

3. **Procesamiento de datos:**

   Luego de formular el problema, la fase siguiente es el procesamiento de datos. Para entrenar modelos de ML precisos, los desarrolladores procesan los datos a fin de convertirlos en un formato utilizable. Entre estos pasos, se incluyen los siguientes:

   Recopilación e integración de datos: garantizan que los datos sin procesar se ubiquen en un lugar con acceso centralizado.

   Procesamiento previo y visualización de datos: transforma los datos sin procesar en un formato comprensible.

   Ingeniería de características: se trata del proceso de creación, transformación, extracción y selección de las variables de los datos.

4. **Desarrollo del modelo:**

   El desarrollo de modelos consiste en su entrenamiento, ajuste y evaluación.

   Se trata de un proceso iterativo que se puede ejecutar muchas veces a lo largo de todo este flujo de trabajo.

   Al principio, tras el entrenamiento, el modelo no arrojará los resultados esperados. Por lo tanto, los desarrolladores realizarán procesos adicionales de ingeniería de características y ajustarán los hiperparámetros del modelo antes de volver a entrenarlo.

5. **Reentrenamiento:**

   Si el modelo no cumple los objetivos empresariales, se deben volver a analizar los datos y las características para identificar formas de mejorar el modelo. La creación de modelos suele ser un proceso iterativo. También puede implicar el ajuste de los hiperparámetros de entrenamiento.

6. **Implementación:**

   Si los resultados son satisfactorios, el modelo se implementa en producción. El modelo ya está listo para hacer predicciones e inferencias con respecto a sí mismo.

7. **Supervisión:**

   El sistema de supervisión del modelo garantiza que este mantenga el nivel deseado de rendimiento mediante la detección temprana y la mitigación. La supervisión también sirve para resolver problemas y comprender el comportamiento del modelo.

8. **Iteraciones:**

   El ciclo de vida de machine learning es un proceso iterativo. El modelo se mejora y refina de forma continua a medida que hay nuevos datos disponibles o que cambian los requisitos. Este carácter iterativo garantiza que el modelo siga siendo preciso y relevante a lo largo del tiempo.

A medida que una empresa sigue estos pasos, es necesario contar con una colaboración fluida entre los diversos roles, como los directores de productos, los desarrolladores, los científicos de datos y los ingenieros. Obtendrá más información sobre este concepto en la sección MLOps.

Caso práctico: centro de llamadas de Amazon.

Hace varios años, Amazon necesitaba mejorar la forma en que derivaba las llamadas de servicio de los clientes, por lo que buscó la ayuda del machine learning.

- **Objetivos empresariales:**

  El sistema original de derivación de llamadas del centro de atención de Amazon funcionaba de esta manera. Un cliente llamaba y lo recibía un menú: "Presione 1 para Devoluciones. Presione 2 para Kindle. Presione 3 para…" y así sucesivamente. Luego, el cliente elegía una opción y lo derivaban a un agente que contaba con los conocimientos necesarios para ayudarlo.

  Durante la fase de formulación del problema de la canalización, Amazon determinó que el sistema de derivación de llamadas de ese entonces era problemático. Amazon vende muchos tipos de productos, por lo que la lista de motivos por los que un cliente puede llamar es casi infinita. Si no le ofrecíamos la opción correcta a un cliente que llamaba, lo podíamos derivar a un generalista o, incluso, al especialista equivocado, quien después tendría que averiguar qué necesitaba el cliente antes de derivarlo, por último, al agente que contaba con los conocimientos adecuados.

  Para algunas empresas, esto puede no ser un problema. Pero para Amazon, que recibe cientos de millones de llamadas de clientes al año, este sistema era ineficiente. Era muy costoso, se perdía mucho tiempo y, lo peor de todo, no les resultaba útil a nuestros clientes para obtener la ayuda que necesitaban.



- **Formulación de problemas:**

  El problema empresarial se centraba en cómo debíamos derivar los clientes a los agentes que contaban con los conocimientos adecuados para reducir las transferencias de llamadas.

  Para ello, necesitábamos predecir con qué conocimiento resolveríamos la llamada de un cliente.

  Al convertirse en un problema de machine learning, el nuevo objetivo consistía en identificar patrones en los datos de los clientes que podríamos usar para predecir cómo debíamos derivar al cliente con precisión. Con base en la formulación de este problema de ML, estaba claro que nos enfrentábamos a un problema de clasificación multiclase.

- **Recopilación e integración de datos:**

  Como queríamos basar nuestras predicciones en datos pasados de las llamadas de servicio al cliente, debíamos tratar con el aprendizaje supervisado. Con el tiempo, entrenaríamos a nuestro modelo con datos históricos de los clientes que incluirían los conocimientos adecuados del agente de clientes o las etiquetas correctas. Luego, el modelo podría hacer sus propias predicciones con datos similares en vistas al futuro. Por ejemplo, predecir que la llamada de un cliente requiere conocimientos sobre Kindle.

  Los datos que necesitábamos procedían de la respuesta a preguntas como: "¿cuáles fueron los pedidos recientes del cliente?", "¿el cliente tenía un Kindle?", "¿es miembro de Prime?". Las respuestas a estas preguntas se convirtieron en características.

- **Procesamiento previo y visualización de datos:**

  Luego, procedimos a la fase de preparación o procesamiento previo de los datos. Esta fase incluye la limpieza y el análisis exploratorio de datos.

  Avanzamos mucho en este punto, pero un ejemplo del análisis de datos consistía en analizar de forma crítica las etiquetas que usábamos. Nos preguntamos: "¿hay alguna etiqueta que debamos excluir del modelo por algún motivo empresarial?", "¿hay alguna etiqueta que no sea del todo precisa?", "¿hay etiquetas que sean lo suficientemente similares como para combinarlas?". Encontrar respuestas a estas preguntas explorando los datos nos ayudaría a reducir la cantidad de características que utilizábamos y a simplificar el modelo.

Un ejemplo de lo que encontramos en este tipo de análisis fue la combinación de etiquetas que representaban varios conocimientos de Kindle en una etiqueta general. De este modo, derivábamos a cada cliente que tenía un problema con un Kindle a un agente que conocía todos los problemas de este producto.

El paso siguiente era visualizar los datos, para lo que probamos varias opciones, como un análisis programático, a fin de obtener una idea rápida de los resúmenes de características y etiquetas. Así pudimos entender mejor los datos con los que trabajábamos. Por ejemplo, descubrimos que el 40 % de las llamadas estaban relacionadas con devoluciones, el 30 % con suscripciones a Prime, el 30 % con Kindle, etc.



- **Entrenamiento de modelos:**

  A fin de prepararse para el proceso de entrenamiento, es importante dividir primero los datos y garantizar una división adecuada de las tareas de entrenamiento y evaluación.

  El objetivo primordial del ML es generalizar más allá de las instancias de datos que se utilizan para entrenar modelos. El modelo se debe evaluar a fin de estimar la calidad de sus predicciones para los datos con los que no se ha entrenado. Sin embargo, como sucede en el aprendizaje supervisado, dado que las instancias futuras tienen valores objetivo desconocidos y no es posible verificar la exactitud de sus predicciones para instancias futuras ahora, usted debe utilizar algunos de los datos para los que ya conoce la respuesta, como un proxy para los datos futuros.

  Evaluar el modelo con los mismos datos que se utilizaron para entrenarlo no es útil porque se recompensa a los modelos que pueden recordar los datos de entrenamiento, en lugar de generalizar a partir de estos.

  Una estrategia común consiste en dividir todos los datos etiquetados disponibles en subconjuntos de entrenamiento, validación y prueba, en general con una proporción del 80 %, 10 % y 10 %. (Otra proporción común es del 70 %, 15 % y 15 %).



Entrenamiento de modelos:

Usó el 80 % de los datos para desarrollar (entrenar) el modelo.

Usó el 10 % de los datos para mejorar el modelo con cada iteración de entrenamiento.

- **Evaluación de modelos:**

  Una vez que estábamos conformes con la forma en que el modelo interactuaba con los datos de prueba que no veía, implementábamos el modelo en producción y lo supervisábamos para garantizar que nuestro problema empresarial realmente se estuviera abordando.

  El problema se basaba en la suposición de que la capacidad de predecir con mayor precisión las habilidades reduciría la cantidad de transferencias que experimentaba un cliente. Esto se puso a prueba después de la implementación, y la cantidad de transferencias disminuyó, lo que se tradujo en una experiencia del cliente mucho mejor.



Evaluación de modelos:

Se utilizó el 10 % de los datos para verificar que el modelo alcanza un rendimiento con la exactitud necesaria o superior:

- ¿Con qué frecuencia se derivaron correctamente las llamadas en el primer intento?
- En promedio, ¿cuántas veces se tuvieron que derivar las llamadas?
- ¿Estos resultados satisfacen nuestras necesidades empresariales?

- **Ajuste de modelos e ingeniería de características:**

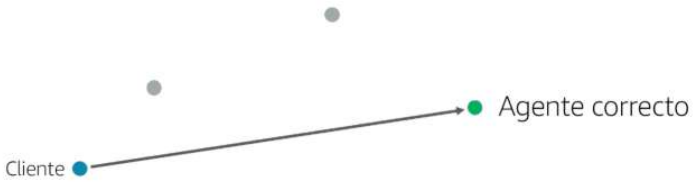
  Después de ejecutar un trabajo de entrenamiento, evaluamos el modelo e iniciamos un proceso de ajustes iterativos en dicho modelo y en los datos.

  Por ejemplo, optimizamos los hiperparámetros. Ajustamos los parámetros de aprendizaje para controlar qué tan rápido o lento aprendía el modelo.

  Si el modelo aprende muy rápido, el algoritmo nunca alcanzará un valor óptimo. Si aprende muy lento, el algoritmo tardará demasiado y puede que nunca converja al nivel óptimo en la cantidad de pasos dada.

Luego, pasamos a la ingeniería de características. Teníamos características que respondían a preguntas como: "¿cuál fue el último pedido de un cliente?", "¿a qué hora fue el último pedido de un cliente?" y "¿el cliente tenía un Kindle?". Cuando ingresamos estas características en el algoritmo de entrenamiento de modelos, solo puede aprender exactamente de lo que le mostramos.

- **Implementación de modelos:**
  Luego, implementamos el modelo que ayuda a derivar a los clientes a los agentes correctos desde el primer momento.



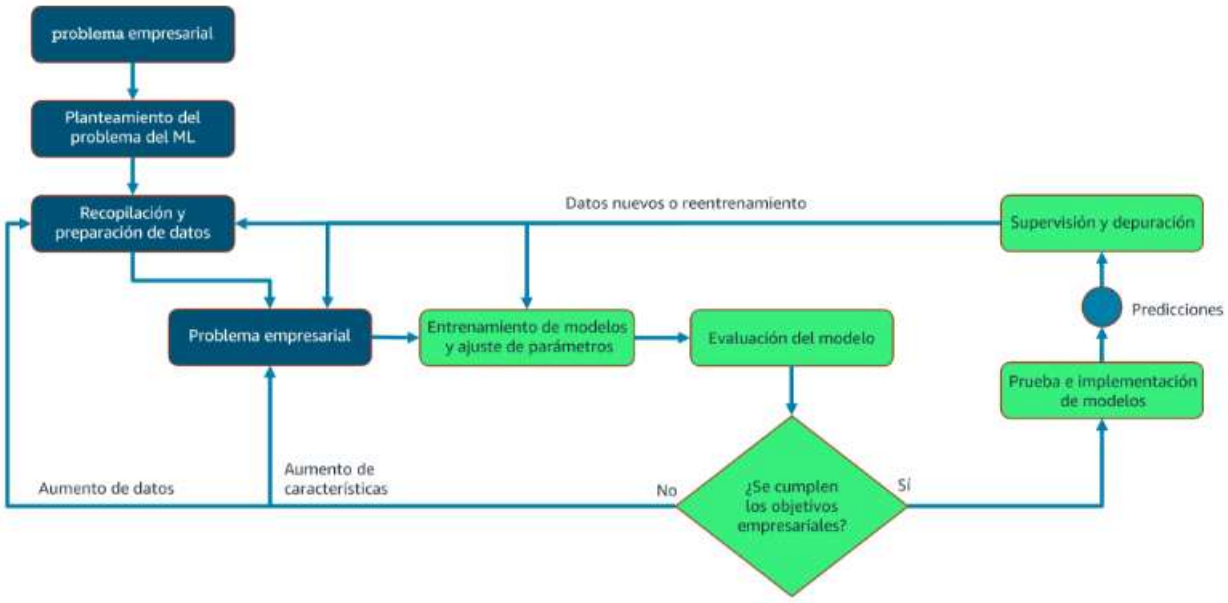
DESARROLLO DE SOLUCIONES DE MACHINE LEARNING CON AMAZON SAGEMAKER AI

Amazon SageMaker AI:

Amazon SageMaker AI es un servicio de machine learning completamente administrado. Puede realizar las siguientes tareas en una única interfaz visual unificada:

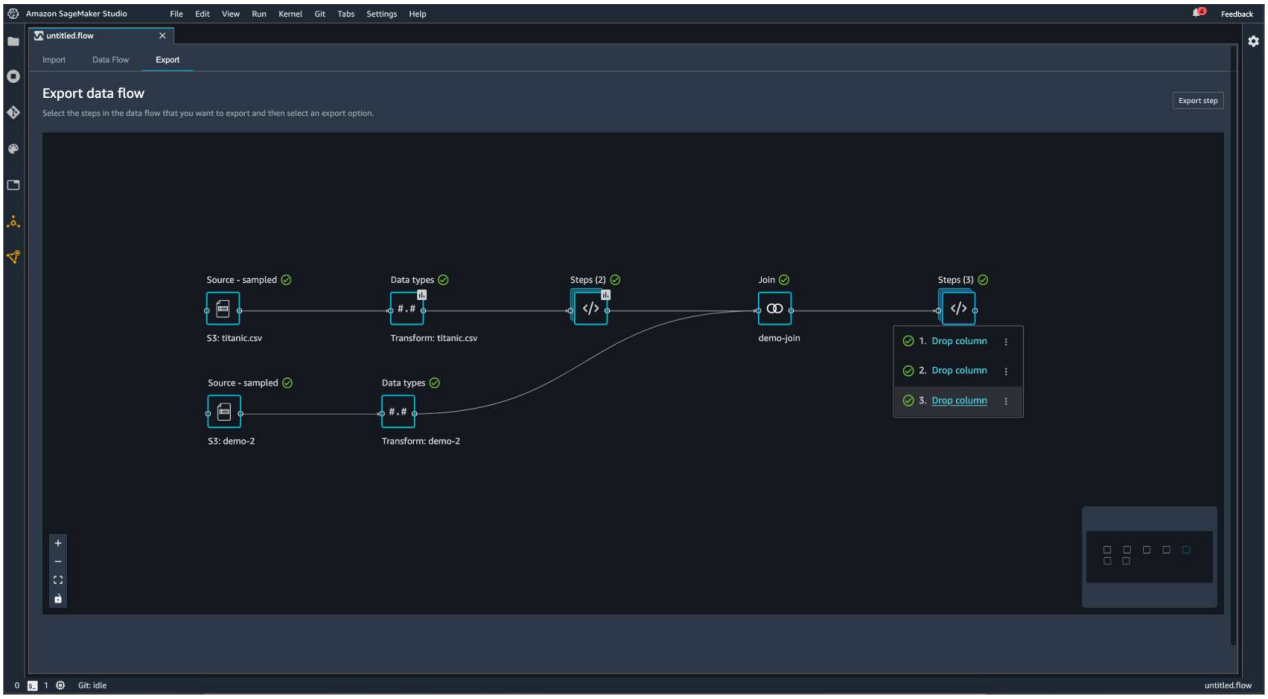
- recopilar y preparar datos,
- crear y entrenar modelos de machine learning, e
- implementar los modelos y supervisar el rendimiento de sus predicciones.

En los siguientes diagramas, se presentan las distintas características de SageMaker AI que puede utilizar en el ciclo de vida de machine learning.



Puede utilizar Amazon SageMaker AI para realizar todos los pasos en un flujo de trabajo de machine learning, desde la recopilación de datos hasta la implementación del modelo.

- **Categoría 1: Recopilación, análisis y preparación de datos.**



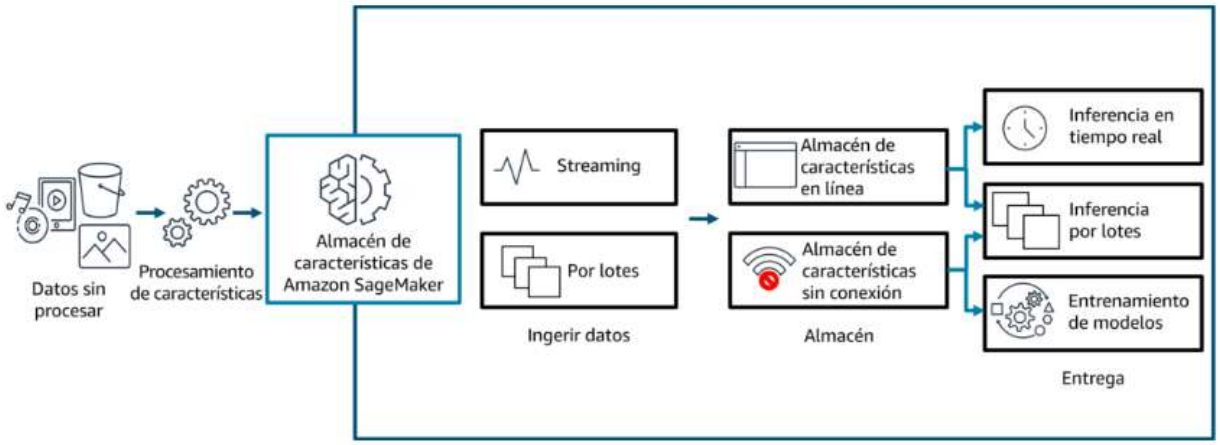
Amazon SageMaker Data Wrangler es una herramienta de poco código o sin código (low-code no-code, LCNC). que ofrece una solución integral para importar, preparar, transformar, presentar y analizar datos mediante una interfaz web. Los clientes pueden agregar sus propios scripts y transformaciones de Python para personalizar los flujos de trabajo.

Para los usuarios más avanzados y para la preparación de datos a gran escala, **Amazon SageMaker Studio Classic** incluye la integración de **Amazon EMR** y las **sesiones interactivas de AWS Glue** a fin de gestionar la preparación interactiva de datos a gran escala y los flujos de trabajo de machine learning en su cuaderno de SageMaker Studio Classic.

Por último, con la **API de procesamiento de SageMaker**, los clientes pueden ejecutar scripts y cuadernos a fin de procesar, transformar y analizar conjuntos de datos. También pueden usar varios marcos de trabajo de ML, como scikit-learn, MXNet o PyTorch, al tiempo que se benefician de entornos de machine learning completamente administrados.

Al final de este paso, los clientes suelen disponer de características para definir el modelo y los datos con los que se entrenará este modelo.

- **Categoría 2: Administración de características.**



Con el **Almacén de características de SageMaker**, los científicos de datos, los ingenieros de machine learning y los profesionales en general pueden crear, compartir y administrar características para el desarrollo del machine learning.

Las características almacenadas en el almacén se pueden recuperar y enriquecer antes de enviarlas a los modelos de ML para realizar inferencias.

- **Categoría 3: Entrenamiento y evaluación de modelos.**

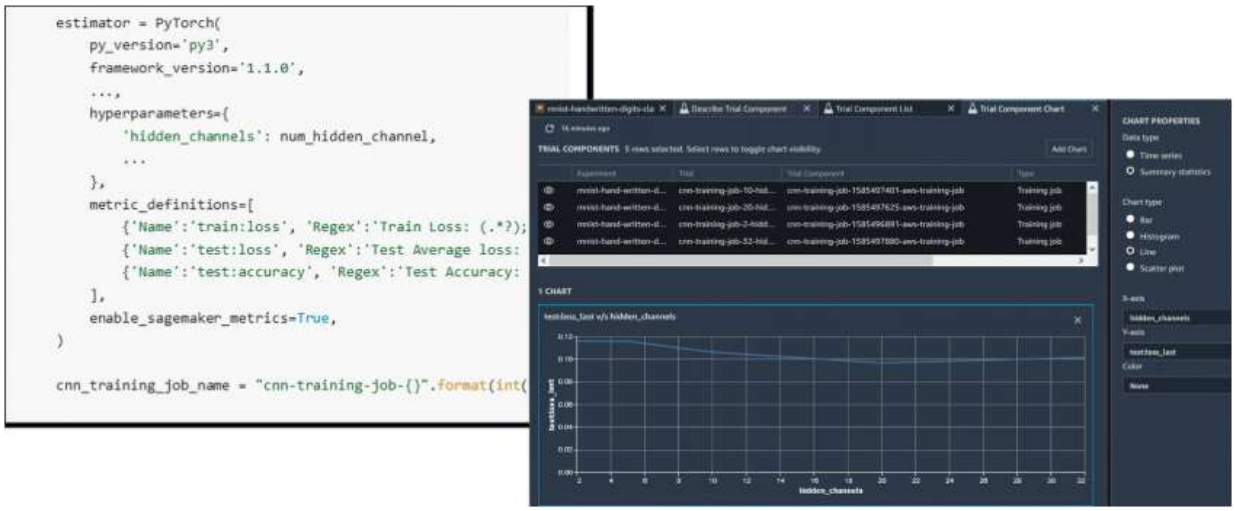
SageMaker AI ofrece una función de trabajo de entrenamiento para el entrenamiento y la implementación de modelos por medio de algoritmos integrados o algoritmos personalizados.

SageMaker AI inicia las instancias de computación de machine learning y utiliza el código de entrenamiento y el conjunto de datos de entrenamiento para entrenar el modelo. También almacena los artefactos resultantes del modelo en un bucket de Amazon Simple Storage Service (Amazon S3) que se puede utilizar más adelante para realizar inferencias.

Los clientes que busquen una opción LCNC pueden utilizar **Amazon SageMaker Canvas.** Con SageMaker Canvas, pueden usar el machine learning para generar predicciones sin tener que escribir código.

Amazon SageMaker JumpStart ofrece modelos de código abierto previamente entrenados que los clientes pueden utilizar para una amplia variedad de tipos de problemas.

- **Categoría 4: Evaluación de modelos.**



Los clientes pueden utilizar **Amazon SageMaker Experiments** para experimentar con varias combinaciones de datos, algoritmos y parámetros, al mismo tiempo que observan el impacto de los cambios progresivos en la exactitud del modelo.

El ajuste de hiperparámetros es una forma de encontrar la mejor versión de los modelos. Para ello, **el Ajuste automático de modelos de Amazon SageMaker** ejecuta muchos trabajos con diferentes hiperparámetros combinados y mide cada uno de ellos con la métrica que usted seleccione.

- **Categoría 5: Implementación.**

Con **SageMaker AI**, los clientes pueden implementar sus modelos de machine learning para hacer predicciones, también conocidas como inferencias. **SageMaker AI** ofrece una amplia selección de opciones de implementación de modelos e infraestructuras de machine learning para ayudar a satisfacer todas sus necesidades de inferencia de machine learning.

- **Categoría 6: Supervisión.**

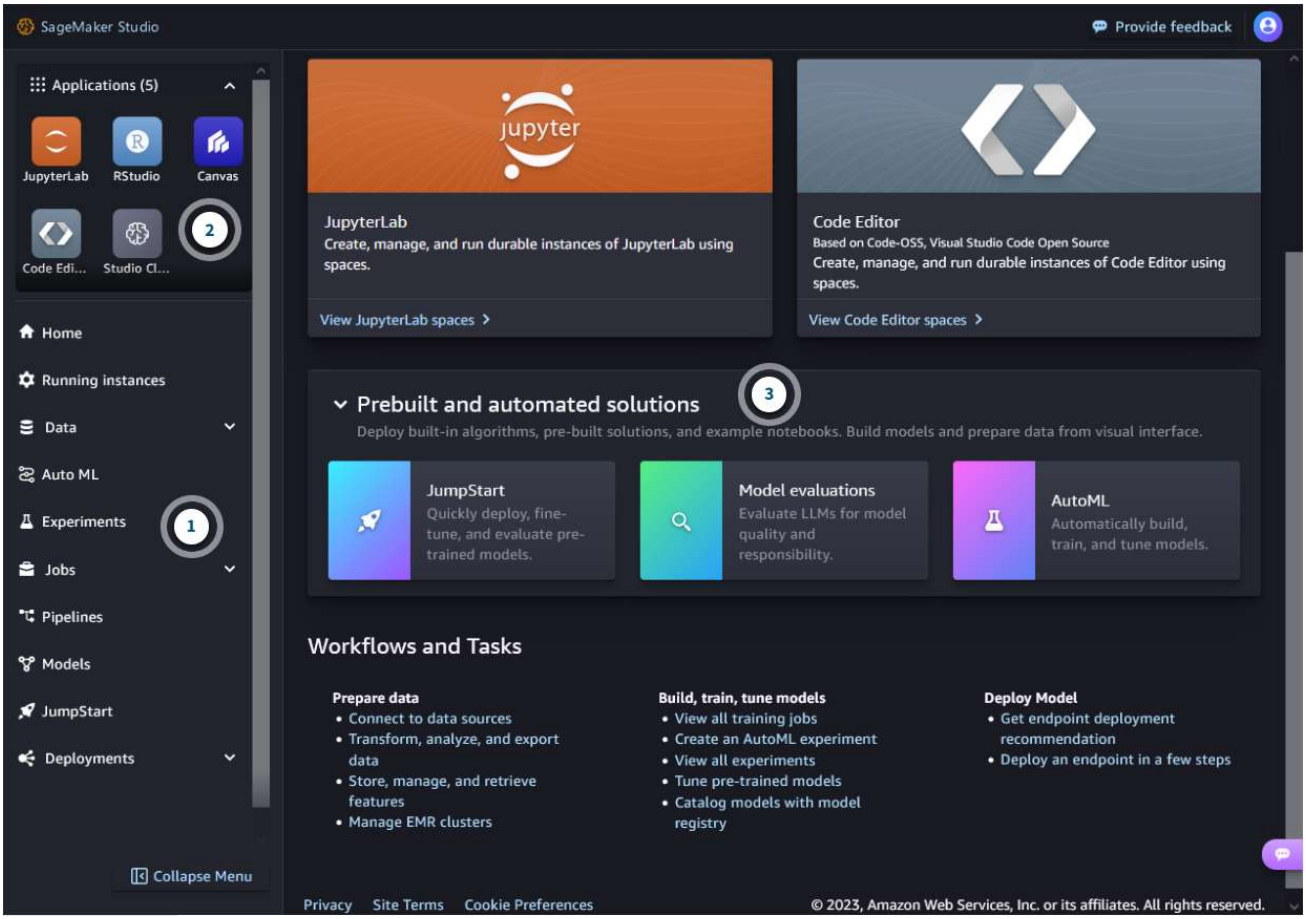
Con el Supervisor de modelos de Amazon SageMaker, los clientes pueden observar la calidad de los modelos de machine learning de SageMaker en producción y configurar una supervisión continua o programada. SageMaker Model Monitor ayuda a mantener la calidad del modelo al detectar infracciones de

los umbrales definidos por el usuario para la calidad de los datos, la calidad del modelo, la desviación de sesgo y desviación de atribución de características.

Entornos de IA de SageMaker:

Amazon SageMaker Studio es la opción recomendada para acceder a SageMaker AI. Se trata de una interfaz de usuario basada en la web que ofrece acceso a todos los entornos y recursos de SageMaker AI.

Para obtener más información al respecto, haga clic en cada uno de los marcadores numerados.



1. **SageMaker Studio:**

   Esta interfaz basada en la web brinda acceso a todas las tareas que puede ejecutar para desarrollar aplicaciones de ML, como preparar datos y entrenar, implementar y supervisar modelos.

2. **Aplicaciones:**

   SageMaker Studio ofrece varias aplicaciones, entre las que se incluyen las siguientes:

   ✓ **JupyterLab:** una herramienta que sirve para desarrollar cuadernos, código y datos de Jupyter

- ✓ **Amazon SageMaker Canvas**: una herramienta de machine learning sin código que sirve para generar predicciones sin necesidad de tener que escribir código
- ✓ **RStudio**: un entorno de desarrollo integrado para el lenguaje R
- ✓ **Editor de código** (basado en Visual Studio Code): otra opción que permite desarrollar código y cuadernos al mismo tiempo que se obtiene acceso a miles de extensiones compatibles con VS Code

3. **ML automatizado:**
   - ✓ **SageMaker JumpStart** ofrece modelos de código abierto previamente entrenados para varios tipos de problemas con el fin de ayudarlo a iniciarse en el machine learning.
   - ✓ **AutoML** se encuentra disponible en **SageMaker Canvas** y simplifica el desarrollo de ML automatizando el proceso de creación e implementación de modelos de machine learning.
   - ✓ **Las evaluaciones de modelos** analizan los modelos de lenguaje de gran tamaño (LLM) y la inteligencia artificial generativa (IA generativa) a fin de determinar la calidad y responsabilidad de los modelos.

FUENTES DE MODELOS DE ML

Implementaciones de modelos:

SageMaker AI admite modelos previamente entrenados, algoritmos integrados e imágenes de Docker personalizadas.

A continuación, se presentan algunas formas de utilizar SageMaker AI para la creación de modelos de machine learning:

- Los modelos previamente entrenados requieren menos esfuerzo y están listos para implementarse o para refinarse e implementarse con **SageMaker JumpStart**.
- Los modelos integrados disponibles en SageMaker AI requieren más esfuerzo y escalan si el conjunto de datos es grande y se necesitan recursos significativos para entrenar e implementar el modelo.
- Si no hay una solución integrada que funcione, intente desarrollar una que use imágenes prediseñadas para machine learning y marcos de trabajo de aprendizaje profundo para los marcos de trabajo compatibles, como scikit-learn, TensorFlow, PyTorch, MXNet o Chainer.
- Puede crear su propia imagen de Docker personalizada que esté configurada para instalar los paquetes o el software necesarios.

Algoritmos integrados de SageMaker AI:

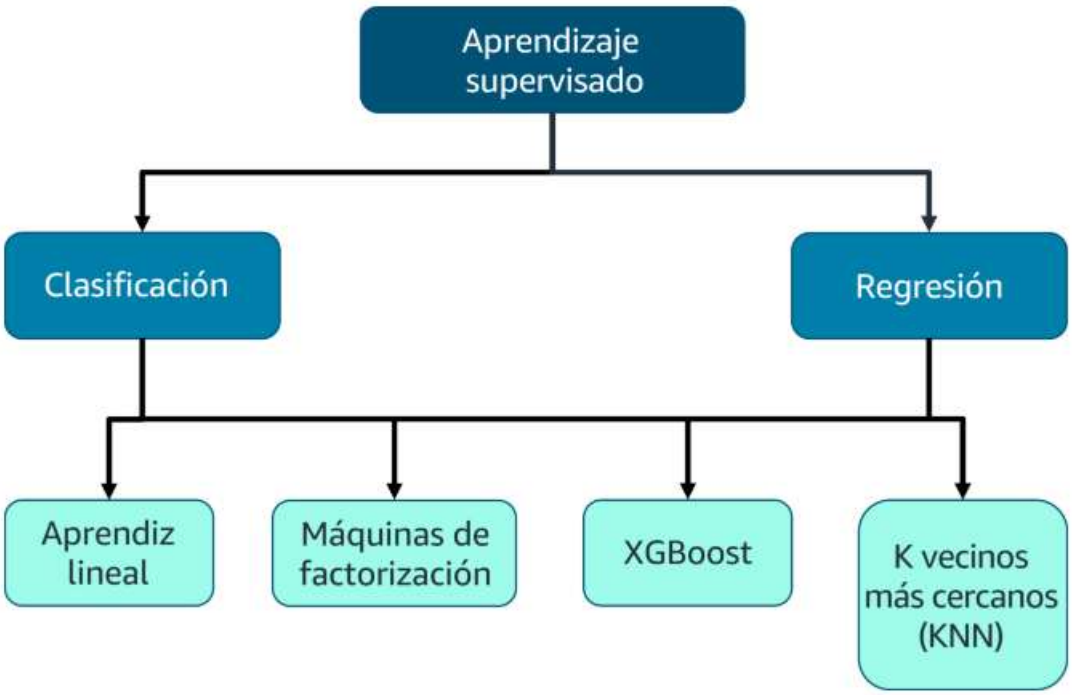
Existen diferentes tipos de algoritmos de machine learning en función de su caso de uso, sus requisitos y sus datos.

- **Introducción:**

  SageMaker AI ofrece algoritmos para diferentes categorías de problemas de machine learning.

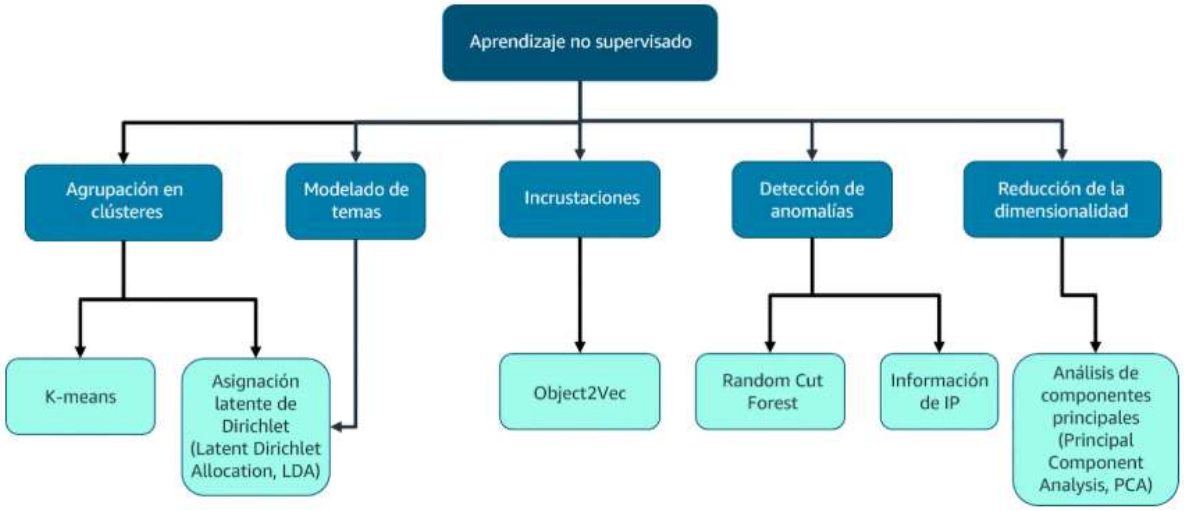
  A continuación, verá una lista breve de los algoritmos más comunes. Para ver una lista completa u obtener más información, consulte la documentación de SageMaker AI.

- **Paso 1: Aprendizaje supervisado con IA.**



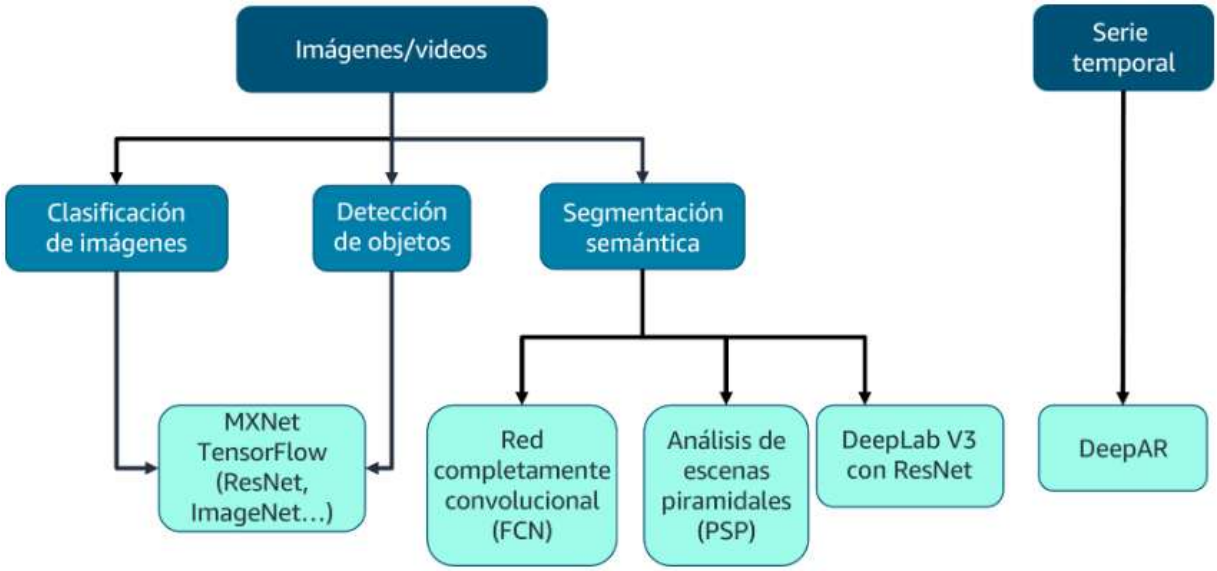
SageMaker AI ofrece varios algoritmos integrados de uso general que puede utilizar para problemas de clasificación o regresión.

- **Paso 2: Aprendizaje no supervisado.**



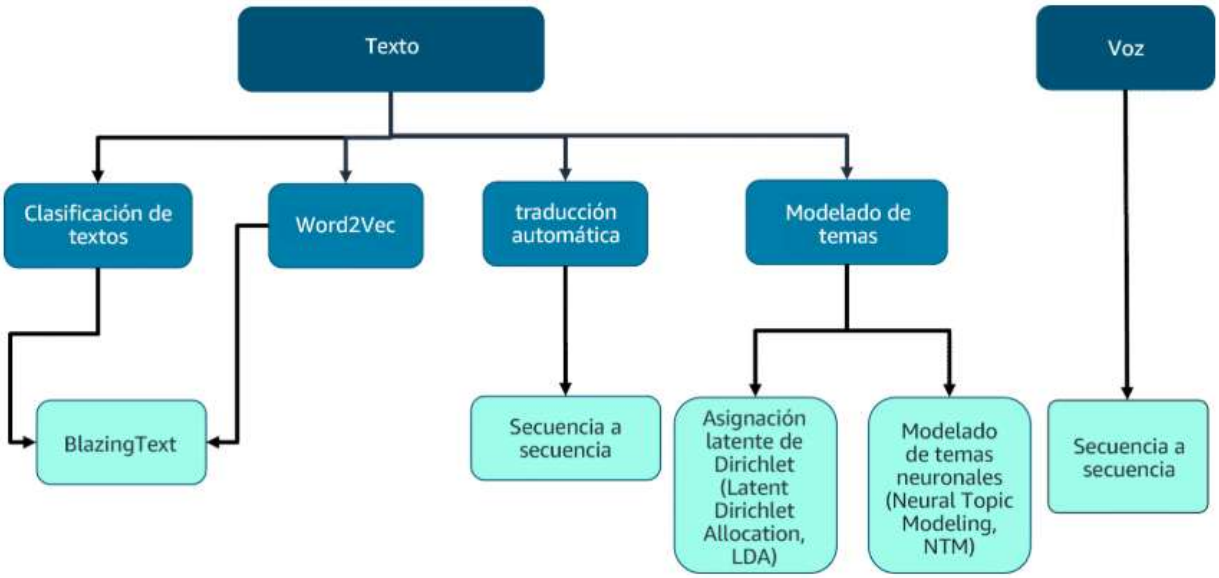
SageMaker AI ofrece varios algoritmos integrados que se pueden utilizar para tareas de aprendizaje no supervisado, como la agrupación en clústeres, la reducción de dimensiones, el modelado de temas con reconocimiento de patrones y la detección de anomalías.

- **Paso 3: Procesamiento de imágenes.**



SageMaker AI también ofrece algoritmos de procesamiento de imágenes que se usan para la clasificación de imágenes, la detección de objetos, la visión artificial y las series temporales.

- **Paso 4: Análisis de texto.**

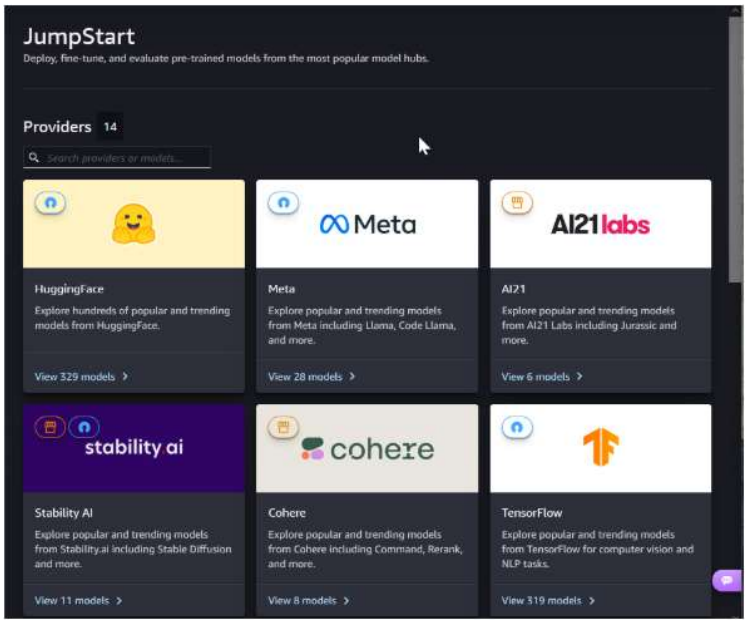


SageMaker AI ofrece algoritmos que se adaptan al análisis de documentos textuales utilizados en el procesamiento del lenguaje natural, la clasificación o el resumen de documentos, el modelado o la clasificación de temas y la transcripción o traducción de idiomas.

SageMaker Jumpstart:

Con SageMaker Jumpstart, puede implementar, ajustar con precisión y evaluar modelos previamente entrenados desde los centros de modelos más populares.

SageMaker JumpStart proporciona modelos de código abierto entrenados previamente de los principales proveedores para varios tipos de problemas con el fin de ayudarlo a iniciarse en el machine learning. Antes de la implementación, puede entrenar y ajustar estos modelos de forma progresiva. SageMaker JumpStart también ofrece plantillas de soluciones que configuran la infraestructura para casos de uso comunes y cuadernos de ejemplos ejecutables para machine learning con SageMaker AI.



Selección de modelos de SageMaker Jumpstart

EVALUACIÓN DEL RENDIMIENTO DE LOS MODELOS DE MACHINE LEARNING

Evaluación de modelos:

En esta sección, aprenderá sobre la evaluación de modelos. Verá qué métricas se pueden usar para dos algoritmos de ML muy comunes, que son el de clasificación y el de regresión.

Conjunto de datos de evaluación de modelos:

La evaluación de los modelos se lleva a cabo después de su entrenamiento. Los datos que utiliza se dividen en tres partes: conjunto de entrenamiento, conjunto de validación y conjunto de prueba. El conjunto de entrenamiento se usa para entrenar el modelo. El conjunto de validación y el de prueba se utilizan para evaluar el rendimiento del modelo entrenado.

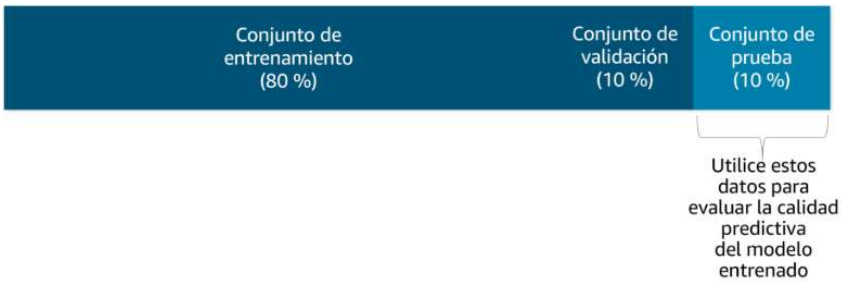
- **Conjunto de validación:**

  Para comenzar a evaluar cómo responde el modelo en un entorno sin entrenamiento, comience analizando los datos que se reservaron como conjunto de validación. Debe asegurarse de que el modelo generalice con los datos que no ha visto. El modelo aún debe mejorarse antes de determinar si está listo para producción.



- **Conjunto de prueba:**

  Una vez que haya mejorado el modelo con esos datos de validación, tendrá todo listo para probarlo por última vez a fin de asegurarse de que su calidad predictiva cumpla sus estándares.

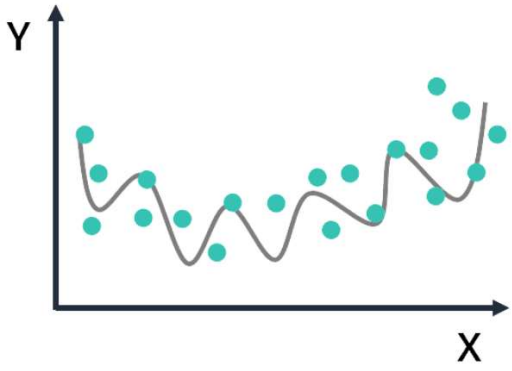


Ajuste de modelo:

El ajuste del modelo es importante para entender la causa raíz de la exactitud ineficiente del modelo, lo que le servirá de guía para tomar medidas correctivas. Para determinar si un modelo predictivo se infraajusta o sobreajusta a los datos de entrenamiento, observe el error de predicción en los datos de entrenamiento y los datos de evaluación.

- **Sobreajuste:**

  El sobreajuste se produce cuando el modelo funciona bien con los datos de entrenamiento, pero no con los datos de evaluación. Esto se debe a que el modelo memorizó los datos que ha visto, pero no puede generalizar con ejemplos que no ha visto.

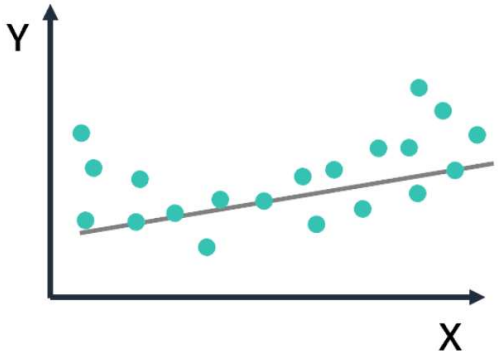


Sobreajuste

- **Infraajuste:**

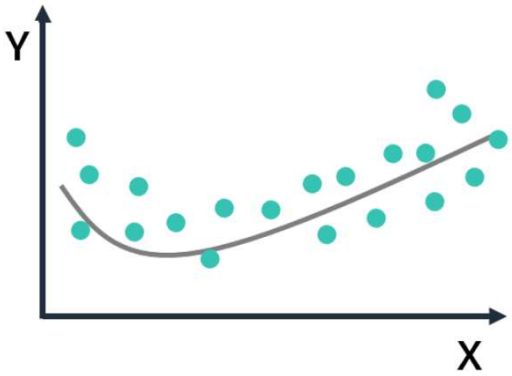
  El infraajuste se produce cuando el modelo tiene un rendimiento deficiente con los datos de entrenamiento. Esto se debe a que el modelo no puede determinar la relación entre los ejemplos de entrada (a veces denominados X) y los valores objetivo (a veces denominados Y).

Si el modelo se infraajusta y tiene un rendimiento deficiente con los datos de entrenamiento, es posible que el modelo sea demasiado simple (las características de entrada no son lo suficientemente expresivas) como para describir bien el objetivo.



Infraajuste

- **Equilibrado:**
  El modelo está equilibrado cuando no está sobreajustado ni infraajustado a los datos de entrenamiento.



Equilibrado

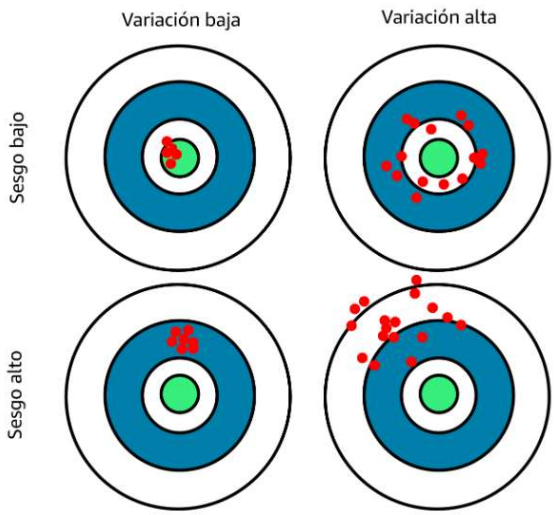
Sesgo y varianza:

Durante la evaluación de modelos, tanto el sesgo como la varianza contribuyen a los errores que cometen con los datos no vistos, lo que afecta su generalización.

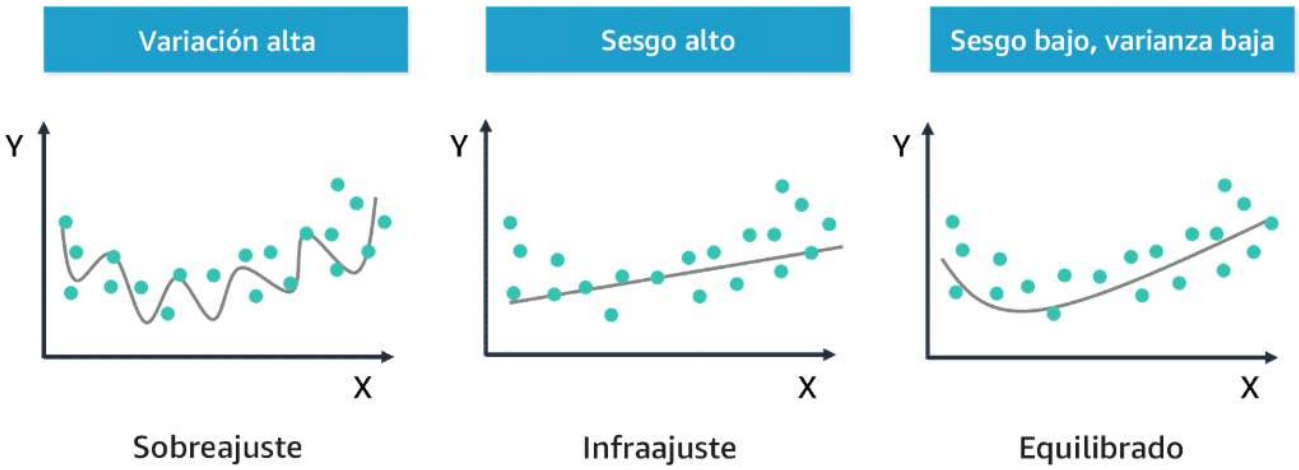
Una diana es una buena analogía porque, en términos generales, el centro de la diana es a donde se apuntan los dardos. En esta situación, el centro de la diana es la etiqueta o el objetivo (predice el valor del modelo), y cada punto es un resultado que el modelo produjo durante el entrenamiento.

Piense en el sesgo como la brecha entre un valor previsto y un valor real, mientras que la varianza describe el grado de dispersión de los valores previstos.

En ML, el algoritmo ideal tiene un sesgo bajo y puede modelar con exactitud la relación real. El algoritmo ideal también tiene una variabilidad baja, ya que produce predicciones consistentes en diferentes conjuntos de datos.



A medida que comience a analizar las métricas de evaluación de forma más específica, tenga en cuenta la compensación entre sesgo y varianza.



Los modelos equilibrados presentan un sesgo y una varianza bajos.

Problemas de clasificación y regresión:

La forma en la que evalúa un modelo de machine learning depende del tipo de problema de ML que esté enfrentando. En esta sección, analizará las métricas de clasificación y regresión.

- **Métricas de clasificación:**
  - ✓ Exactitud
  - ✓ Precisión
  - ✓ Exhaustividad
  - ✓ F1
  - ✓ AUC-ROC
- **Métricas de regresión:**
  - ✓ Error cuadrático medio
  - ✓ R al cuadrado

Métricas de problemas de clasificación:

Ejemplo de clasificación:

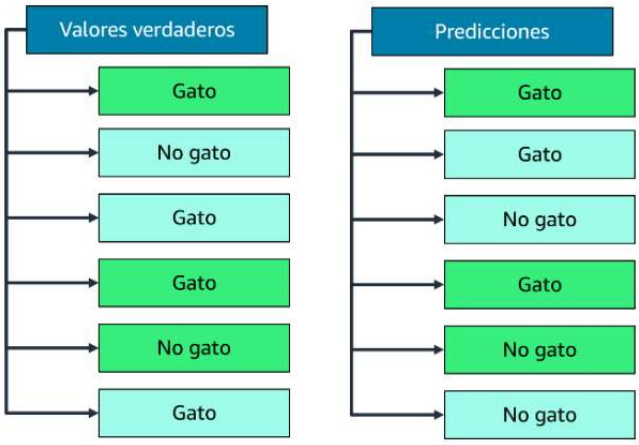
El siguiente es un problema de clasificación binaria en el que un modelo de reconocimiento de imágenes etiqueta datos como “gato” o “no gato”.

Para evaluar un problema de clasificación como el que se muestra, siga estos pasos:

Paso 1: envíe las observaciones del modelo cuyos valores objetivo conoce.

Paso 2: compare las predicciones que devuelve el modelo con el valor objetivo conocido.

Paso final: calcule una métrica de resumen que muestre cuánto coinciden los valores reales y los previstos.

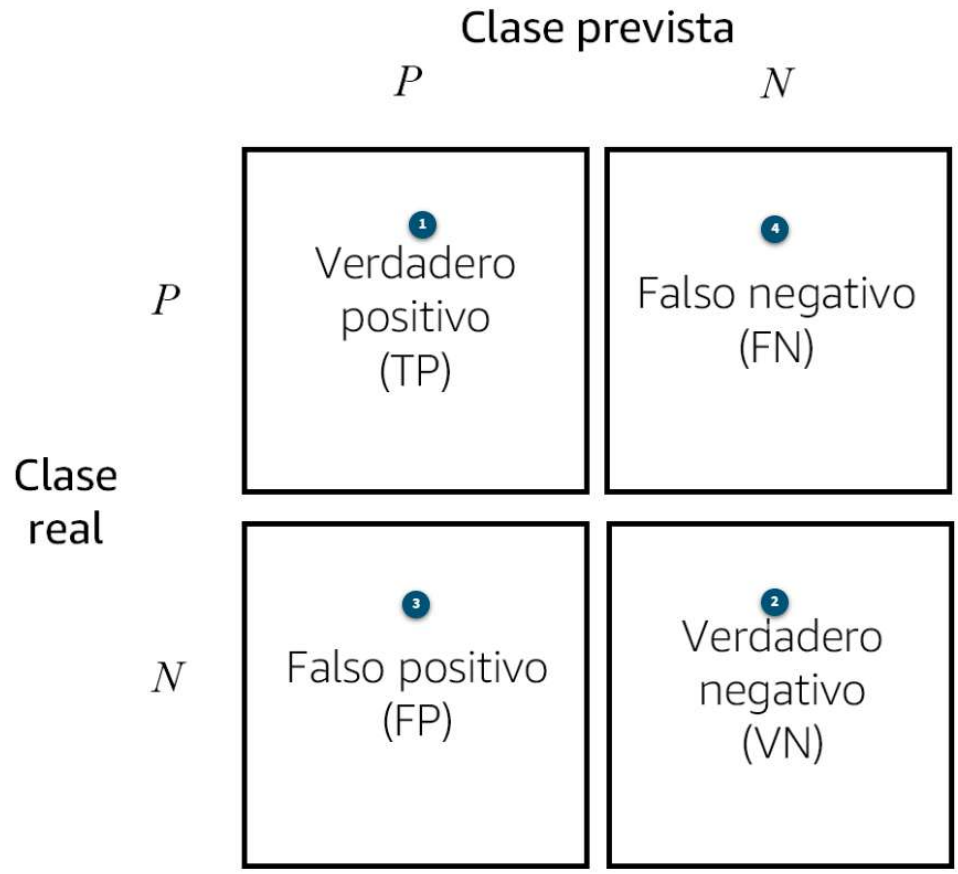


Ejemplo de un problema simple de clasificación binaria

Matriz de confusión:

Una matriz de confusión puede servir para clasificar por qué y cómo se equivoca un modelo y es esencial para ejecutar estos tipos de evaluaciones de modelos para problemas de clasificación. Examine el siguiente gráfico, que es una matriz de confusión para el ejemplo de reconocimiento de imágenes. La matriz ofrece una comparación general de cómo coincidieron las clases previstas con las clases reales.

Una vez que se haya aplicado el modelo a los datos de prueba, cada uno de los cuatro recuadros de la matriz incluirá un número total de las incidencias únicas de verdaderos positivos, falsos positivos, falsos negativos y verdaderos negativos.



1. **Verdadero positivo (TP):**

   Si la etiqueta o clase real es “gato”, que se identifica como “P” (positivo) en la matriz de confusión, y la etiqueta o clase prevista también es “gato”, se obtiene un resultado verdadero positivo, que es un buen resultado para el modelo.

2. **Verdadero negativo (TN):**

   Del mismo modo, si tiene una etiqueta real de “no gato”, que se identifica como “N” (negativo) en la matriz de confusión, y la etiqueta o clase prevista también es “no gato”, se obtiene un verdadero negativo, que también es un buen resultado para el modelo. En ambos casos, el modelo predijo el resultado correcto cuando usó los datos de prueba.

3. **Falso positivo (FP):**

   Esta no es la situación ideal y ocurre cuando la clase real es negativa, es decir, “no gato”, pero la clase prevista es positiva, es decir, “gato”. Esto se denomina “falso positivo” porque la predicción es positiva, pero incorrecta.

4. **Falso negativo (FN):**

   Esto tampoco es lo ideal. Un falso negativo ocurre cuando la clase real es positiva, es decir, “gato”, pero la clase prevista es negativa, es decir, “no gato”.

Exactitud:

Para calcular la exactitud del modelo, también conocida como su “puntuación”, debe sumar las predicciones correctas y, luego, dividir ese número por la cantidad total de predicciones.

$$\frac{TP + TN}{(TP + TN + FP + FN)}$$

Cálculo de la exactitud de un modelo

Aunque la exactitud es una métrica que se usa ampliamente para los problemas de clasificación, tiene límites. Esta métrica es menos eficaz cuando hay muchos casos de verdadero negativo en el conjunto de datos. Por esta razón, suelen utilizarse otras dos métricas en estas situaciones: la precisión y la exhaustividad.

Precisión:

La precisión elimina las predicciones negativas de la imagen. La precisión es la proporción de predicciones positivas que realmente son correctas. Para calcularla, divida el recuento de verdaderos positivos por la cantidad total de positivos.

$$\frac{TP}{TP + FP}$$

Cálculo de la precisión

Cuando el costo de los falsos positivos es alto en su situación empresarial particular, la precisión puede ser una métrica útil. Piense en un modelo de clasificación que identifica los correos electrónicos que son spam y los que no lo son. En este caso, necesita que el modelo no etiquete un correo electrónico legítimo como spam e impida que los usuarios vean ese correo electrónico.

Exhaustividad:

Además de la precisión, también existe la exhaustividad (o sensibilidad). En la exhaustividad, se observa la proporción de conjuntos correctos que se identifican como positivos. Para calcular la exhaustividad, se divide el recuento de verdaderos positivos por la suma de verdaderos positivos y falsos negativos. Observando esa proporción, se obtiene una idea de qué tan bueno es el algoritmo para detectar, por ejemplo, gatos.

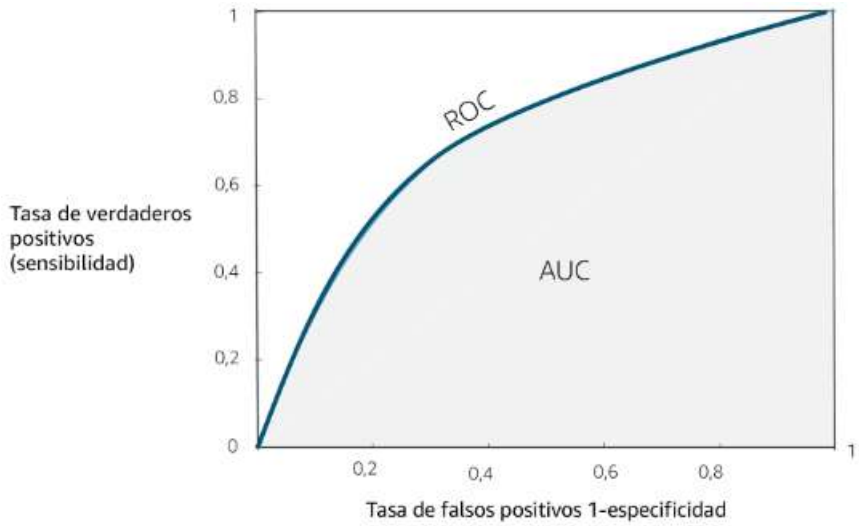
$$\frac{TP}{TP + FN}$$

Cálculo de la exhaustividad

Piense en un modelo que deba predecir si un paciente tiene una enfermedad terminal o no. En este caso, el uso de la precisión como métrica de evaluación no tendrá en cuenta los falsos negativos del modelo. Para que el modelo resulte exitoso, es extremadamente importante y fundamental que no dé resultados falsos negativos. Un falso negativo no identificaría que un paciente tiene una enfermedad terminal cuando en realidad sí la tiene. En esta situación, la exhaustividad sería una métrica más adecuada.

AUC-ROC:

El área bajo la curva de característica operativa del receptor (AUC-ROC) es otra métrica de evaluación. La ROC es una curva de probabilidad, mientras que la AUC representa el grado o la medida de separabilidad.

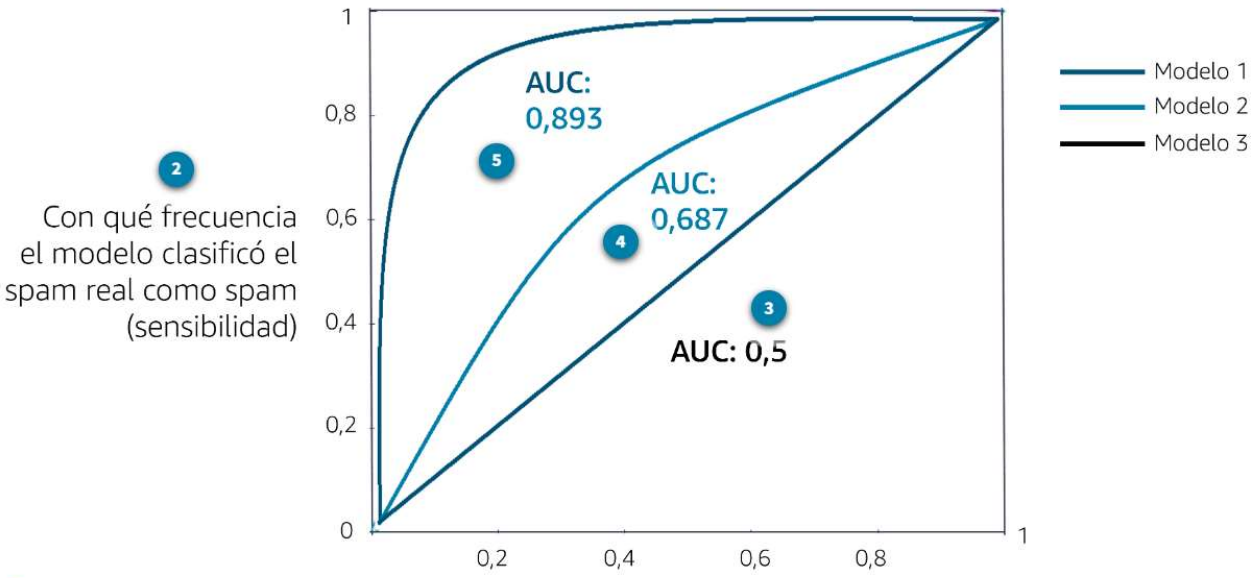


AUC-ROC usa la sensibilidad (tasa de verdaderos positivos) y la especificidad (tasa de falsos positivos)

En general, AUC-ROC puede mostrar cómo se ve la curva de verdaderos positivos frente a falsos positivos en varios umbrales. Esto significa que, cuando calcula la curva AUC-ROC, traza varias matrices de confusión en diferentes umbrales y las compara entre sí para averiguar qué umbral necesita para su caso práctico empresarial.

Ejemplo: Clasificación de correos electrónicos spam.

Tomemos un ejemplo de clasificación de correos electrónicos spam. Los correos electrónicos se clasifican según la puntuación de riesgo del clasificador. En el siguiente grafo, los correos electrónicos con puntuación alta se encuentran a la izquierda, y la gran mayoría de los correos electrónicos con puntuación baja se encuentran a la derecha.



1. **Eje X:**

   En el eje X, se muestra el porcentaje de correos electrónicos legítimos que se verán afectados por la acción (en este caso, dejar los correos electrónicos en la carpeta de spam).

2. **Eje Y:**

   En el eje Y, se muestra el porcentaje de spam que está capturando.

3. **Situación negativa:**

   Cuando el AUC es de alrededor de 0,5, el modelo equivale a hacer conjeturas aleatorias.

4. **Punto de compensación:**

   El ángulo en la curva es un excelente punto de compensación. En este punto, existe un buen equilibrio entre la población buena a la que afecta y la mala que captura. Por supuesto, si marginar una población y capturar a la población mala tiene diferentes costos, puede desplazar ese punto de operación hacia la izquierda o hacia la derecha. Los buenos clasificadores producen buenas curvas de compensación.

5. **Curva de compensación perfecta:**

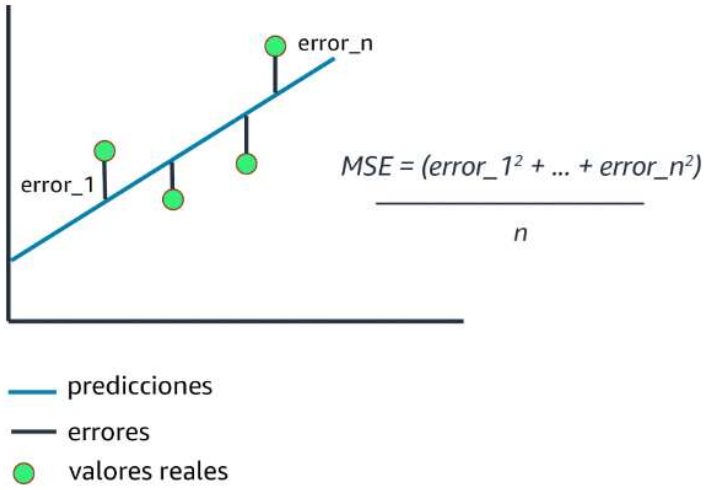
   A medida que mejora las características que utiliza como entradas para el clasificador, o a medida que mejora el algoritmo, la curva se desplaza hacia arriba y hacia la izquierda. La curva de compensación perfecta es aquella que se desplaza a lo largo de la parte superior izquierda del rectángulo.

Métricas de los problemas de regresión:

En caso de un problema de regresión, hay otras métricas comunes que puede usar para evaluar el modelo, como el error cuadrático medio y la R cuadrada. El error cuadrático medio se usa con mucha frecuencia.

Error cuadrático medio:

El propósito general del error cuadrático medio (mean squared error, MSE) es medir la exactitud de la predicción para evaluar el rendimiento del modelo de regresión. Usted determina la predicción a partir del modelo y compara la diferencia entre la predicción y el resultado real.



Cálculo del error cuadrático medio

Más específicamente, usted toma la diferencia entre la predicción y el valor real, eleva esa diferencia al cuadrado, suma todas las diferencias al cuadrado de todas las observaciones y, a continuación, divide por el número total de observaciones a fin de obtener el promedio.

Cuanto más pequeño sea el MSE, mejor será la exactitud predictiva del modelo.

R al cuadrado:

R cuadrado es otra métrica que habitualmente se utiliza con los problemas de regresión lineal. R cuadrado explica la fracción de variación representada por el modelo. Es como un porcentaje, que informa un número de 0 a 1. Cuando R al cuadrado se aproxima a 1, por lo general indica que gran parte de la varianza de los datos puede explicarse por el propio modelo.

El MSE se centra en el error cuadrático promedio de las predicciones del modelo para proporcionar una medida del rendimiento del modelo. R al cuadrado proporciona una medida del grado de ajuste del modelo a los datos. Ambas son importantes, pero ofrecen perspectivas diferentes.

Métricas empresariales:

En la sección anterior, aprendió a evaluar el rendimiento de un modelo de ML. Pero recuerde que, al iniciar un proyecto, los objetivos empresariales y los KPI establecidos son las métricas que se utilizan para evaluar si se cumplen los objetivos.

Para validar y supervisar el rendimiento del modelo, establezca métricas numéricas que se relacionen directamente con los KPI. Estos KPI se establecen en la fase de identificación de los objetivos empresariales y pueden incluir ciertos objetivos, como aumentar las ventas o disminuir los costos o la pérdida de clientes.

Evalúe si las métricas de rendimiento reflejan con precisión la tolerancia de la empresa ante los errores. Por ejemplo, los falsos positivos pueden generar costos de mantenimiento excesivos en los casos de uso del mantenimiento predictivo. Otro ejemplo es decidir si adquirir un cliente nuevo es más costoso que retenerlo. Una empresa debe centrarse en las métricas numéricas, como la precisión y la exhaustividad, a fin de diferenciar los requisitos empresariales y estar más alineada con el valor empresarial.

Considere la posibilidad de desarrollar métricas personalizadas que ajusten el modelo directamente a los objetivos empresariales. Una forma consiste en desarrollar una función de costos para evaluar el impacto económico del modelo. Para la función de costos, puede especificar el costo o el valor de las predicciones correctas y el costo de los errores.

Mediante las pruebas A/B o la técnica de implementación canario, los desarrolladores pueden experimentar con dos o más variantes de un modelo y alcanzar los objetivos empresariales.

IMPLEMENTACIÓN DE MODELOS

Tipos de implementación de modelos:

La implementación de modelos es la integración del modelo y sus recursos en un entorno de producción a fin de que pueda usarse para crear predicciones.

- **API autoalojada:**

  En un enfoque de API autoalojada, usted implementa y aloja sus modelos de ML en su propia infraestructura, ya sea en las instalaciones o en la nube (mediante máquinas virtuales o contenedores). Este enfoque implica configurar y administrar la infraestructura necesaria, como servidores web, equilibradores de carga y bases de datos, para entregar los modelos de ML como API.

- **API administrada:**

  Los servicios de API administrados son servicios basados en la nube que brindan un entorno completamente administrado para implementar y alojar sus modelos de ML como API. SageMaker AI es uno de ellos. Estos servicios abstraen la administración de la infraestructura subyacente para que pueda centrarse en crear e implementar sus modelos.

Las ventajas de las API autoalojadas incluyen un mayor control de la infraestructura, posibles ahorros de costos (según el uso) y la capacidad de personalizar el entorno de implementación. Sin embargo, este enfoque requiere más gastos operativos generales y responsabilidad para la administración y el mantenimiento de la infraestructura.

La elección de un servicio de API administrado o una API autoalojada para la implementación de ML depende de factores como los requisitos específicos de su caso de uso, el nivel de control y personalización requeridos, los recursos y la experiencia disponibles, y las consideraciones de costo.

SageMaker AI:

SageMaker AI es un servicio de machine learning completamente administrado. Con SageMaker AI, los científicos de datos y los desarrolladores pueden crear, entrenar e implementar modelos de machine learning de forma rápida y segura en un entorno alojado listo para la producción. En unos pocos pasos, puede implementar un modelo en un entorno seguro y escalable.

SageMaker AI proporciona lo siguiente:

- Implementación con un clic o una sola llamada a la API
- escalado automático,
- servicios de alojamiento de modelos, y
- Puntos de enlace HTTPS que puedan alojar varios modelos



Puede utilizar SageMaker AI para implementar un modelo y obtener predicciones de varias formas.

- **En tiempo real:**

  La inferencia en tiempo real es ideal para las cargas de trabajo de inferencia que tienen requisitos de latencia baja, interactivos y de tiempo real.

- **Transformación por lotes:**

  Use la transformación por lotes cuando necesite obtener inferencias de grandes conjuntos de datos, pero no necesite un punto de conexión persistente. También puede usarlo cuando necesite preprocesar conjuntos de datos para eliminar el ruido o el sesgo que interfiera con el entrenamiento o la inferencia del conjunto de datos.

- **Asincrónica:**

  La inferencia asincrónica es una función de SageMaker AI que pone en cola las solicitudes entrantes y las procesa de forma asincrónica. Esta opción es ideal para solicitudes con grandes tamaños de carga (hasta 1 GB), tiempos de procesamiento extensos (hasta una hora) y requisitos de latencia casi en tiempo real.

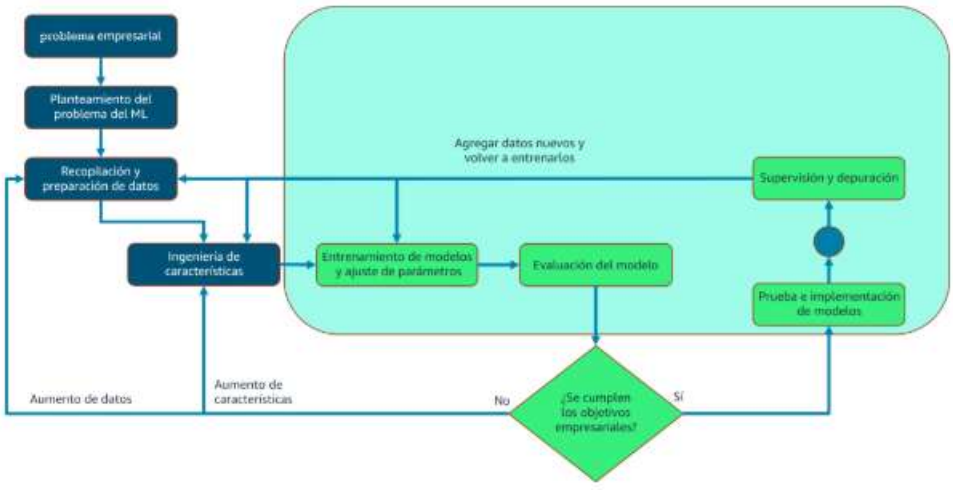
- **Sin servidor:**

  La inferencia sin servidor bajo demanda es ideal para las cargas de trabajo que tienen períodos de inactividad entre los aumentos de tráfico y pueden tolerar el arranque en frío. Se trata de una opción de inferencia diseñada específicamente que puede utilizar para implementar y escalar modelos de ML sin configurar ni administrar la infraestructura subyacente.

CONCEPTOS FUNDAMENTALES DE LAS MLOps:

MLOps:

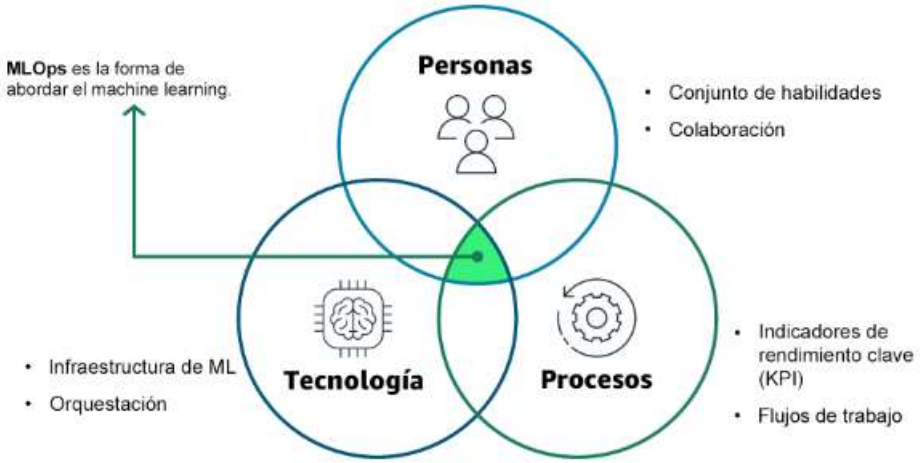
Las MLOps combinan personas, tecnología y procesos a fin de ofrecer soluciones de ML colaborativas.



Las MLOps abarcan desde el desarrollo de modelos hasta su supervisión.

“MLOps” hace referencia a la práctica de operacionalizar y optimizar el ciclo de vida integral del machine learning, desde el desarrollo y la implementación de modelos hasta su supervisión y mantenimiento. Además, garantizan que los modelos no solo se desarrollen, sino que también se implementen, supervisen y vuelvan a entrenar de manera sistemática y reiterada.

Se trata de una extensión de los principios y prácticas de DevOps al dominio específico de los sistemas de machine learning.



Al igual que las DevOps, se basan en un enfoque colaborativo y optimizado del ciclo de vida del desarrollo de machine learning. Es la intersección de las personas, los procesos y la tecnología lo que optimiza las actividades integrales necesarias para desarrollar, crear y operar cargas de trabajo de machine learning.

Uso de MLOps:

Las aplicaciones que exponen modelos entrenados pueden tener diferentes requisitos y estrategias de alojamiento que las aplicaciones estándar. Los modelos entrenados son sensibles a los cambios en los datos; por lo tanto, una aplicación basada en modelos que funcionó bien cuando se implementó por primera vez podría no funcionar tan bien días, semanas o meses luego de su implementación. Para tener en cuenta estas diferencias, se necesitan diferentes procesos y procedimientos para las aplicaciones basados en la administración de ML.

Las MLOps tienen en cuenta los aspectos singulares de los proyectos de inteligencia artificial y machine learning (IA/ML) en la administración de proyectos, la integración y entrega continuas (CI/CD) y el control de calidad. Con estas, puede mejorar el tiempo de entrega, reducir los defectos y hacer que la ciencia de datos sea más productiva.

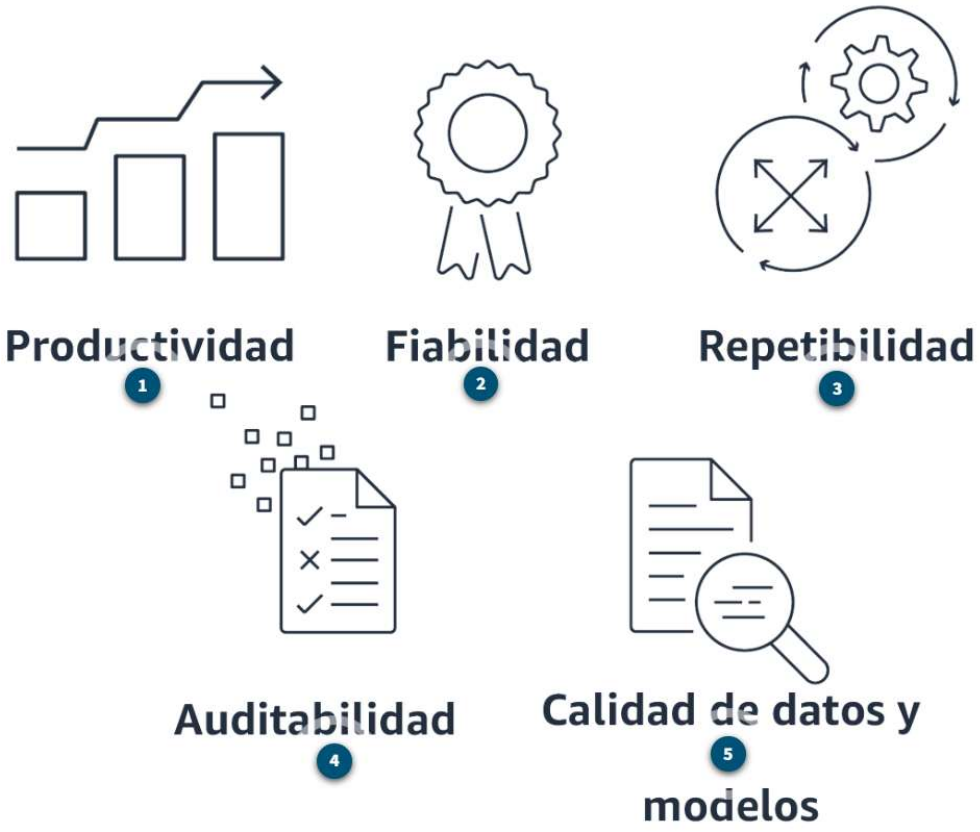
Objetivos de las MLOps:

Uno de los objetivos de las MLOps es enviar cargas de trabajo de ML a producción y mantenerlas en funcionamiento. Para ello, las MLOps adoptan muchos principios y prácticas de las DevOps relacionadas con el desarrollo, el entrenamiento, la implementación, la supervisión y el reentrenamiento de modelos de machine learning. El objetivo es usar las MLOps para lo siguiente:

- Aumentar el ritmo del ciclo de vida de desarrollo de modelos mediante la automatización,
- Mejorar las métricas de calidad a través de pruebas y supervisión.
- Promover una cultura de colaboración entre científicos de datos, ingenieros de datos, ingenieros de software y operaciones de TI.
- Proporcionar transparencia, explicabilidad, audibilidad y seguridad de los modelos mediante la gobernanza de modelos.

Beneficios de las MLOps:

La adopción de prácticas de MLOps brinda un tiempo de comercialización más rápido para los proyectos de ML mediante los siguientes beneficios.



1. **Productividad:**

   Al proporcionar entornos de autoservicio con acceso a conjuntos de datos seleccionados, los ingenieros y los científicos de datos pueden avanzar más rápido y perder menos tiempo con datos faltantes o no válidos.

2. **Fiabilidad:**

   Al incorporar prácticas de CI/CD, los desarrolladores son capaces de implementar procesos con mayor calidad, rapidez y consistencia.

3. **Repetibilidad:**

   Al automatizar todos los pasos del ciclo de vida del desarrollo del machine learning, puede garantizar que el proceso sea repetible, incluida la forma en que se entrena, evalúa, versiona e implementa el modelo.

4. **Auditabilidad:**

   Al aplicar el control de versiones en todas las entradas y salidas, desde los experimentos de ciencia de datos y los datos de origen hasta los modelos entrenados, puede demostrar exactamente cómo se creó el modelo y dónde se implementó.

5. **Calidad de datos y de modelos:**

   Las MLOps le permiten aplicar políticas que brindan protección contra el sesgo del modelo y que realizan un seguimiento de los cambios en las propiedades estadísticas de los datos y de la calidad del modelo a lo largo del tiempo.

Principios clave de las MLOps:

Entre los principios clave de las MLOps, se incluyen los siguientes:

- **Control de versiones:**

  A fines de reproducibilidad, los flujos de trabajo de machine learning deben realizar un seguimiento de los cambios en los activos, como los datos, el código y los modelos. El control de versiones se puede revertir a versiones anteriores cuando sea necesario y,

  en general, tanto el control de versiones como la revisión del código brindan un machine learning fiable y reproducible.

- **Automatización:**

  Para garantizar la repetibilidad, la consistencia y la escalabilidad, usted puede automatizar las distintas etapas de la canalización de machine learning. Esto incluye la ingesta de datos, el procesamiento previo, el entrenamiento de modelos y las etapas de validación e implementación.

Las pruebas automatizadas sirven para detectar problemas con anticipación, con lo que puede corregir errores y aprender con rapidez.

- **CI/CD:**

  Mediante la automatización, puede probar e implementar activos de forma continua de las siguientes maneras:

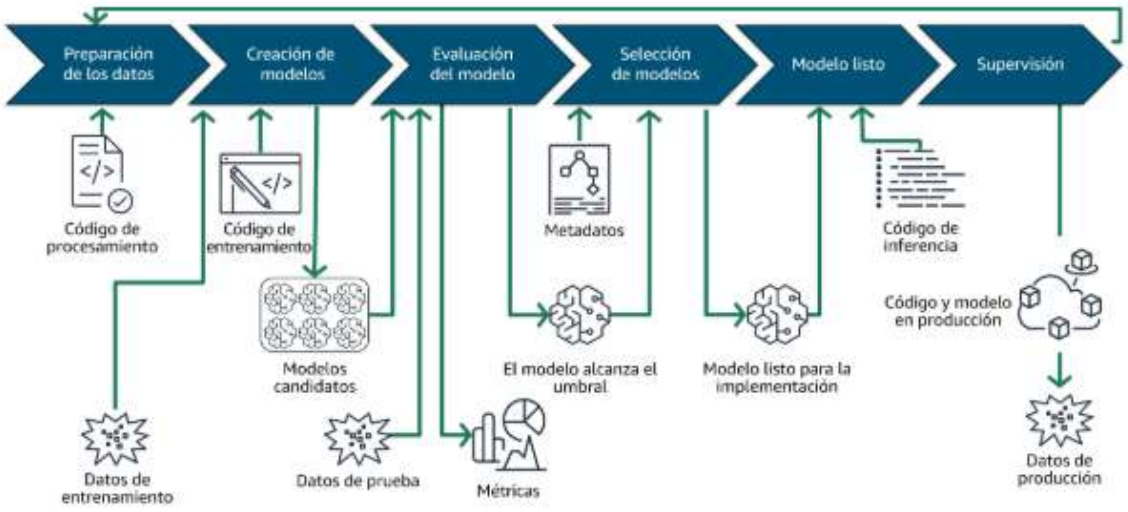
  - ✓ La integración continua extiende la validación y la prueba del código a los datos y modelos de la canalización.
  - ✓ La entrega continua implementa automáticamente el modelo recién entrenado o el servicio de predicción de modelos.
  - ✓ El entrenamiento continuo vuelve a entrenar automáticamente los modelos de ML para su reimplementación.
  - ✓ La supervisión continua utiliza la supervisión de datos y modelos para analizar las métricas empresariales.

- **Gobernanza del modelo:**

  Una buena gobernanza de los sistemas de machine learning requiere una estrecha colaboración entre los científicos de datos, los ingenieros y las partes interesadas de la empresa. Contar con documentación clara, canales de comunicación eficaces y mecanismos de comentarios permite que todas las personas estén en sintonía y que los modelos mejoren con el tiempo. También es crucial proteger la información confidencial y el acceso, y respetar las normas de cumplimiento. Un proceso estructurado de revisión, validación y aprobación de modelos previo a su implementación verifica la equidad, el sesgo y la ética. La gobernanza administra todos los aspectos de los sistemas para garantizar la eficiencia.

El ciclo de vida del ML y las MLOps:

La mayoría de las cargas de trabajo de ML implican la administración de código, datos y modelos.



Ciclo de vida del ML

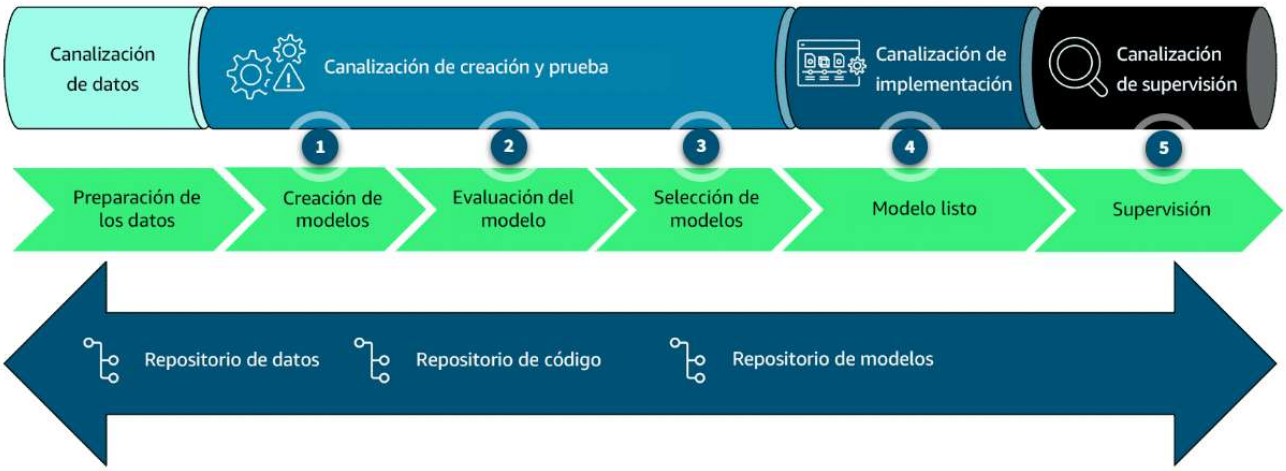
La administración del código, los datos y los modelos a lo largo del ciclo de vida del ML requiere los siguientes puntos de contacto:

- código de procesamiento en la preparación de datos,
- datos y código de entrenamiento para la creación de modelos,
- modelos candidatos, pruebas y datos de validación en la evaluación de modelos,
- metadatos durante la selección del modelo,
- modelos listos para la implementación y código de inferencia durante la implementación, y
- código de producción, modelos y datos para la supervisión.

A través de las MLOps, se operacionalizan los procesos relacionados con el desarrollo, la implementación, la supervisión y la gobernanza de los modelos de ML.

Implementación de MLOps:

El siguiente diagrama es un ejemplo de un proceso de automatización integral. Los ciclos de vida de ML producidos suelen contener canalizaciones de entrenamiento e implementación independientes.



1. **Creación del modelo:**

   La canalización de creación de modelos crea nuevos modelos al momento de iniciarse, por ejemplo, cuando hay nuevos datos disponibles.

2. **Evaluación de modelos:**

   Una vez completada la canalización de creación de modelos, puede aplicar medidas de control de calidad en la fase de registro del modelo. La fase de control de calidad puede ser manual (intervención humana en el bucle) o automática.

   Si un modelo cumple las métricas básicas de rendimiento, puede registrarse en un catálogo de modelos.

3. **Aprobación del modelo:**

   El registro se puede utilizar para aprobar o rechazar versiones del modelo. La acción de aprobación del modelo puede actuar como la iniciación para poner en marcha la canalización de implementación.

4. **Implementación de modelos:**

   La canalización de implementación es la más similar a los sistemas de CI/CD tradicionales. Esta canalización incluye pasos como los siguientes:

   - ✓ Fuente
   - ✓ compilación;
   - ✓ implementación al entorno de almacenamiento provisional;
   - ✓ pruebas;
   - ✓ promoción al entorno de producción.

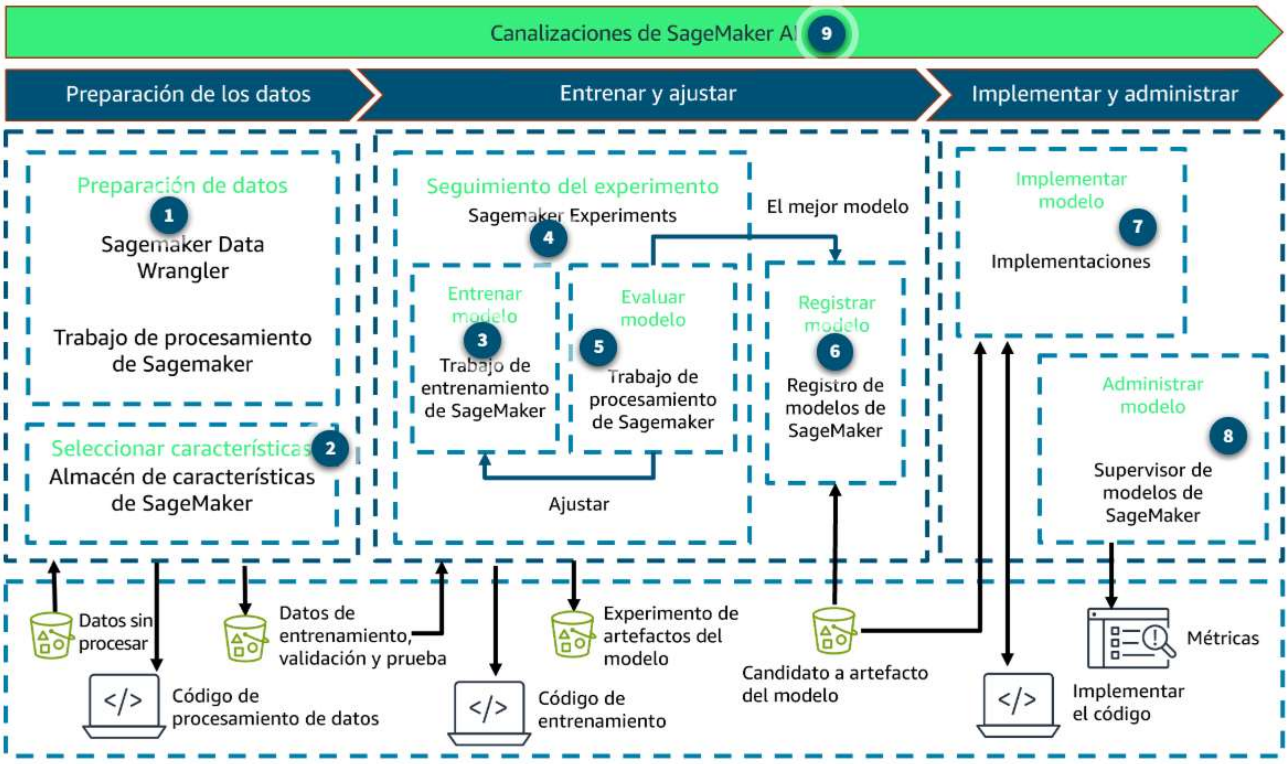
5. **Modelo en producción:**

   Tan pronto como el modelo esté en producción, usted debería recibir comentarios del sistema en vivo. En el caso de las soluciones de ML, supervise la infraestructura de alojamiento, la calidad de los datos y el rendimiento de los modelos.

Los niveles de implementación de las MLOps pueden variar según las organizaciones y los proyectos.

Servicios de AWS para las MLOps:

En el siguiente diagrama, verá los servicios de AWS que pueden usarse para implementar una canalización de MLOps.



1. **Preparación de datos:**

   SageMaker Data Wrangler es una herramienta LCNC que ofrece una solución integral para la importación, la preparación, la transformación, la creación de características y el análisis de datos a través de una interfaz web.

   AI utilizar la API de procesamiento de IA de SageMaker, los científicos de datos pueden ejecutar guiones y cuadernos para procesar, transformar y analizar conjuntos de datos en varios marcos de

machine learning, como scikit-learn, MXNet o PyTorch, al tiempo que se benefician de los entornos de machine learning completamente administrados.

2. **Almacenamiento de características:**

   Con el almacén de características de SageMaker, los científicos de datos, los ingenieros de machine learning y los profesionales en general pueden crear, compartir y administrar características para el desarrollo del ML.

3. **Entrenamiento:**

   SageMaker AI ofrece una función de trabajo de entrenamiento para entrenar modelos mediante algoritmos integrados o algoritmos personalizados.

4. **Experimentos:**

   Use SageMaker Experiments para experimentar con varias combinaciones de datos, algoritmos y parámetros, mientras observa el impacto de los cambios progresivos en la exactitud del modelo.

5. **Trabajo de procesamiento:**

   El procesamiento de SageMaker AI se refiere a las capacidades para ejecutar tareas de preprocesamiento y posprocesamiento de datos, ingeniería de características y evaluación de modelos en la infraestructura completamente administrada de SageMaker AI.

6. **Registro:**

   Con el registro de modelos de SageMaker, puede catalogar modelos, administrar sus versiones, gestionar su estado de aprobación e implementarlos en producción.

7. **Implementaciones:**

   Con SageMaker AI, puede implementar sus modelos de machine learning a fin de hacer predicciones, también conocidas como inferencias. SageMaker AI proporciona una amplia selección de opciones de implementación de modelos e infraestructuras de machine learning para ayudar a satisfacer todas sus necesidades de inferencia de machine learning.

8. **Supervisión de modelos:**

   Con Model Monitor de SageMaker, puede supervisar la calidad de los modelos de machine learning de SageMaker en producción.

9. **Canalizaciones:**

   Puede utilizar las canalizaciones de creación de modelos de Amazon SageMaker para crear flujos de trabajo integrales que administren e implementen trabajos de SageMaker AI.